

OpenDoPE Specification

Version 3.0 · Working Draft · 18 September 2026

Abstract

OpenDoPE (Open Document Processing Ecosystem) is a specification for generating Word documents from XML data. It builds on content control data binding as defined in ISO/IEC 29500. It adds what that standard leaves out: conditional inclusion of content, repetition of content for each item in a list, inclusion of other documents, and binding of rich content such as XHTML, images and Word fragments.

The instructions live in content control tags and in small custom XML parts. The user’s own XML is left untouched and can follow any schema. This version revises the OpenDoPE conventions v2.3 to describe the behaviour of the reference implementation in docx4j 17.1.1, including the features added since v2.3.

Status

This document is a Working Draft. It supersedes the conventions page for version 2.3 published at opendope.org. Where the two differ, this document describes the reference implementation. **Error! Reference source not found.** summarises the differences.

Editor: Jason Harrop, Plutext Pty Ltd. Comments are welcome through the docx4j project on GitHub.

Contents

Abstract	1
Status	1
1 Introduction	3
1.1 Overview	3
1.2 Scope	3
1.3 Terminology	3
1.4 Normative References	5
1.5 Informative References	5
2 Package Structure	5
2.1 Parts	5
2.2 Selecting the Data Part	6
3 Tag Syntax	7
4 The XPath Part	7
5 Processing Model	9

6	Data Binding	10
6.1	Resolving the Value	10
6.2	Plain Text Values	11
6.3	Hyperlinks	11
6.4	Typed Content Controls	11
7	Conditions	12
7.1	The Conditions Part	12
7.2	Boolean Conversion	13
7.3	Effect of a Condition	14
8	Repeats	14
8.1	Expanding a Repeat	14
8.2	Contextualising Expressions	15
8.3	Position Conditions	16
8.4	Word Repeating Sections	17
9	Rich Content	17
9.1	XHTML	17
9.2	Word Fragments	18
9.3	Pictures	19
10	Components	19
11	After Binding	20
11.1	Integrity Repair	20
11.2	Finishing	20
11.3	Removing Content Controls and Parts	20
11.4	Reverting	21
11.5	Updating Data from the Document	21
12	Security Considerations	22
13	Conformance	22
	Appendix A Tag Parameter Reference (Normative)	23
	Appendix B docx4j Implementation Notes (Informative)	24
	B.1 Running the Pipeline	24
	B.2 Configuration Properties	25
	B.3 Extension Points	25
	Appendix C Changes from Version 2.3 (Informative)	26
	Appendix D Acknowledgements (Informative)	27
	Appendix E Revision History (Informative)	28

1 Introduction

1.1 Overview

Generating documents from a template and data is a common requirement. Search and replace on the document surface is fragile, because Word splits text across runs. It also cannot include, exclude or repeat whole paragraphs or table rows. Approaches based on merge fields and bookmarks share these limits.

Content controls provide a better foundation [ISO29500]. A content control (w:sdt) can carry a w:dataBinding that points by XPath into a custom XML part. Word keeps the control and the XML element synchronised in both directions. The XML can follow whatever schema the business uses.

The standard does not say how to make content conditional or repeated, how to include one document in another, or how to bind anything richer than plain text. OpenDoPE fills these gaps with a small convention:

- A content control's w:tag names its role, for example od:condition=c1 or od:repeat=x2.
- The referenced definitions live in dedicated custom XML parts: the XPaths part, the Conditions part and the Components part.
- A processor evaluates the definitions against the data part and writes an instance document that any WordprocessingML consumer can open.

Because the data part is not altered by the conventions, one template can be processed any number of times with different data.

EXAMPLE An invoice template contains a table row wrapped in a repeat content control, and a bank details paragraph wrapped in a condition content control. Processing it against an invoice with three items yields a table with three rows, and includes the bank details only when the data says so.

1.2 Scope

This specification defines the OpenDoPE parts, the tag vocabulary, and the processing model for data binding, conditions, repeats, rich content and components. It also defines the optional steps that follow binding: integrity repair, finishing transformations, content control removal, and reverting an instance document to its template.

The question-and-answer layer used for interactive document assembly is described only in outline. Authoring tools, such as Word add-ins that create OpenDoPE markup, are out of scope.

1.3 Terminology

1.3.1 Key Words

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **NOT RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

1.3.2 Definitions

Authoring Tool

Software that creates or edits a template, writing its content control tags and OpenDoPE parts.

Binding Step

The processing step that writes data values into bound content controls (Section **Error! Reference source not found.**).

Consumer

An application, such as Microsoft Word, that opens an instance document.

Content Control

A `w:sdt` element at block, run, row or cell level.

Data Part

The custom XML part holding the user's data, against which XPath expressions are evaluated. Version 2.3 calls it the answer file.

Instance Document

The document produced by processing a template against data.

Occurrence

One appearance of a repeat content control in the document being processed. A repeat control that has been copied and pasted has two occurrences.

OpenDoPE Part

A custom XML part whose root element is in a namespace beginning `http://opendope.org/`.

Preprocessing Step

The processing step that evaluates conditions and expands repeats (Sections **Error! Reference source not found.** and **Error! Reference source not found.**).

Processor

Software that transforms a template into an instance document according to this specification. docx4j is the reference processor.

Repeat Base

The repeat XPath after normalisation, used to contextualise the XPaths of content inside a repeat.

Repeat Instance

One copy of a repeat content control, corresponding to one node selected by the repeat XPath.

Store Item ID

The GUID in a custom XML part's properties part (`ds:datastoreItem/@ds:itemID`) that identifies the part.

Template

A WordprocessingML package containing OpenDoPE-tagged content controls and the OpenDoPE parts they refer to.

1.4 Normative References

- [ISO29500]** ISO/IEC 29500-1:2016, “Information technology — Document description and processing languages — Office Open XML File Formats — Part 1: Fundamentals and Markup Language Reference”.
- [MS-DOCX]** Microsoft Corporation, “[**MS-DOCX**]: Word Extensions to the Office Open XML (.docx) File Format”.
- [RFC2045]** Freed, N. and N. Borenstein, “Multipurpose Internet Mail Extensions (MIME) Part One: Format of Internet Message Bodies”, RFC 2045, November 1996.
- [RFC2119]** Bradner, S., “Key words for use in RFCs to Indicate Requirement Levels”, BCP 14, RFC 2119, March 1997.
- [RFC8174]** Leiba, B., “Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words”, BCP 14, RFC 8174, May 2017.
- [XHTML1]** W3C, “XHTML 1.0 The Extensible HyperText Markup Language (Second Edition)”, W3C Recommendation, 1 August 2002.
- [XPath1]** Clark, J. and S. DeRose, “XML Path Language (XPath) Version 1.0”, W3C Recommendation, 16 November 1999.

1.5 Informative References

- [DOCX4J]** Plutext Pty Ltd, “docx4j”, version 17.1.1, <https://github.com/plutext/docx4j>.
- [IMPORTXHTML]** Plutext Pty Ltd, “docx4j-ImportXHTML”, version 17.1.1, <https://github.com/plutext/docx4j-ImportXHTML>.
- [OPENDOPE23]** Harrop, J., “OpenDoPE: Sdt Content Control conventions for repeats & conditionals”, version 2.3, https://opendope.org/opendope_conventions_v2.3.html.
- [XPath2]** Berglund, A., et al., “XML Path Language (XPath) 2.0 (Second Edition)”, W3C Recommendation, 14 December 2010.

2 Package Structure

2.1 Parts

A template is a WordprocessingML package **[ISO29500]**. Besides the main document part it contains the custom XML parts in the table below. Each has a custom XML properties part that assigns its store item ID.

Table 1 – Custom XML parts of a template

Part	Root element	Namespace	Present
Data part	any	any	Always

Part	Root element	Namespace	Present
XPaths part	xpaths	http://opendope.org/xpaths	When any <code>od:</code> tag refers to an XPath
Conditions part	conditions	http://opendope.org/conditions	When <code>od:condition</code> is used
Components part	components	http://opendope.org/components	When <code>od:component</code> is used
Questions part	questionnaire	http://opendope.org/questions	Interactive assembly only
Answers part	answers	http://opendope.org/answers	Optional standardised data format

REQ-001 A processor **MUST** identify an OpenDoPE part by the namespace and local name of its root element, not by its part name.

REQ-002 Each custom XML part of a template **MUST** be the target of a `customXml` relationship from the main document part.

NOTE Word silently discards custom XML parts that are not related from the main document part.

2.2 Selecting the Data Part

A template may contain several custom XML parts. The processor needs to know which holds the data, both to evaluate expressions and to replace it with runtime data. The following rules apply in order.

1. If the XPaths part exists and has entries, use the part named by the first entry whose store item ID resolves to a data part. Skip entries whose prefix mappings declare the cover page properties, document management properties or core properties namespaces.
2. If there is no XPaths part, use the store item ID of the first `w:dataBinding` found in the main document body, with the same exclusions.
3. Otherwise, if the package contains exactly one custom XML part that is neither an OpenDoPE part nor a well-known Microsoft part, use it. The well-known parts are cover page properties, document management properties, core properties and bibliography.
4. Otherwise the data part is undetermined.

REQ-003 A processor **MUST NOT** choose between several candidate parts under the third rule.

REQ-004 A processor **MUST** report an error when it is asked to bind and the data part is undetermined.

REQ-005 A processor **MUST** compare store item IDs case-insensitively.

Two store item IDs address the package's document properties rather than a custom XML part. `{6C3C8BC8-F283-45AE-878A-BAB7291924A1}` addresses the core properties part and `{6668398D-A668-4E3E-A5EB-62B293D839F1}` the extended properties part. Word uses `{55AF091B-3C7A-`

41E3-B477-F2FDAA23CFDA} for the cover page properties part, which is an ordinary custom XML part.

REC-001 A processor **SHOULD** evaluate expressions bound to the core or extended properties store item IDs against the corresponding document properties part.

3 Tag Syntax

OpenDoPE instructions are carried in `w:sdtPr/w:tag/@w:val`. The value is a list of parameters in query string form:

```
tag    = param *( "&" param )
param  = key [ "=" value ]
key    = 1*( %x21-25 / %x27-3C / %x3E-7E ) ; printable, no "&" or "="
value  = *( %x20-25 / %x27-7E ) ; anything but "&"
```

In the document part XML the separator is written `&`. Keys are percent-decoded, with `+` decoded as a space. Values are taken literally and may contain `=`. A parameter without `=` is treated as a key whose value is the key itself.

EXAMPLE `od:repeat=x2&od:finish=t_r` marks a repeat whose XPath is entry x2, and asks the finisher to apply template `t_r` to each instance.

REQ-006 A processor **MUST** ignore parameters whose keys it does not recognise.

REQ-007 A template **MUST NOT** give one content control more than one of `od:condition`, `od:repeat` and `od:component`.

Parameters outside the `od:` vocabulary let authoring tools keep their own data in the tag, such as a display name. **Error! Reference source not found.** lists every parameter this specification defines.

NOTE Word limits the length of a tag. Version 2.3 cites 64 characters. Referring to definitions by short ID keeps tags within the limit.

REQ-008 A processor **MUST NOT** evaluate `od:condition` or `od:repeat` on a content control that has a `w:dataBinding`, except for a Word repeating section (Section **Error! Reference source not found.**).

4 The XPaths Part

The XPaths part lists the XPath expressions used by the template. Tags refer to entries by ID, which keeps tags short and lets one expression serve several controls.

```
<xpaths xmlns="http://opendope.org/xpaths" version="3.0" booleanConversion="xpath2"
  authoringTool="Example Author" authoringToolVersion="1.4">
  <xpath id="x1">
    <dataBinding storeItemID="{8B049945-9DFE-4726-9DE9-CF5691E53858}"
      xpath="/invoice[1]/customer[1]/name[1]"/>
  </xpath>
  <xpath id="x2">
    <dataBinding storeItemID="{8B049945-9DFE-4726-9DE9-CF5691E53858}"
      xpath="/invoice[1]/items[1]/item"/>
  </xpath>
</xpaths>
```

The root element records how the template was authored. Each `xpath` entry holds one expression.

Table 2 – Attributes of the XPaths root element

Attribute	Use	Meaning
@version	required	The version of this specification the template was authored against, for example 3.0.
@booleanConversion	required	The boolean conversion mode the expressions were written for (Section Error! Reference source not found.).
@authoringTool	optional	The name of the authoring tool that last saved the template.
@authoringToolVersion	optional	The version of that authoring tool.

REQ-009 A template **MUST** declare the version of this specification it was authored against in `xpaths/@version`.

The declaration records what the template was written for. It is not a claim that the template conforms. A template with no declaration was authored against version 2.3 or earlier.

REQ-010 A processor **MUST** refuse to process a template that declares a version newer than the one it implements.

REQ-011 Whenever an authoring tool saves a template, it **MUST** set `xpaths/@authoringTool` and `xpaths/@authoringToolVersion` to identify itself and its version, replacing any values written by another tool.

REQ-012 A processor **MUST** keep the root element's attributes when it rewrites the XPaths part (Section **Error! Reference source not found.**).

NOTE Knowing which tool last saved a template, and which version of the specification it targeted, is the first question when a template misbehaves in a newer tool or processor.

Table 3 – Attributes of an XPaths entry

Attribute	Use	Meaning
@id	required	Identifier of type <code>xs:ID</code> , referenced from tags and conditions.

Attribute	Use	Meaning
<code>dataBinding/@xpath</code>	required	The XPath expression.
<code>dataBinding/@storeItemID</code>	required	Store item ID of the part the expression is evaluated against.
<code>dataBinding/@prefixMappings</code>	optional	Namespace prefix declarations, written as in <code>w:dataBinding</code> , for example <code>xmlns:ns0='urn:example'</code> .
<code>@questionID</code>	optional	The question that elicits this value in interactive assembly.
<code>@name</code> , <code>@description</code> , <code>@comments</code>	optional	Documentation for template authors.
<code>@type</code> , <code>@required</code> , <code>@prepopulate</code> , <code>@fieldWidth</code> , <code>@lower</code> , <code>@lowerOperator</code> , <code>@upper</code> , <code>@upperOperator</code>	optional	Data type and range constraints for interactive tools.
<code>@source</code>	optional	Where a library entry came from.

REQ-013 Each `xpath/@id` **MUST** be unique within the XPaths part.

REQ-014 An expression **MUST** be valid XPath 1.0 **[XPATH1]**, unless the processor has been configured for a later XPath version.

NOTE A template written for the XPath 2.0 boolean conversion mode (Section **Error! Reference source not found.**) needs an XPath 2.0 processor, and its expressions may use XPath 2.0 syntax.

An expression that selects a single element can also appear in a `w:dataBinding` on the content control, so that Word keeps the control live. An expression that yields a string, number or boolean cannot. For example, `count(/invoice/items/item)` is an *extended bind*. Only the `od:xpath` tag refers to it, and its value is fixed when the document is processed.

REC-002 Where a content control has both a `w:dataBinding` and an `od:xpath` tag, the two **SHOULD** specify the same expression and store item ID.

Preprocessing adds entries for the contextualised expressions it creates inside repeats (Section **Error! Reference source not found.**). The rewritten XPaths part is saved in the instance document.

NOTE docx4j logs duplicate identifiers and keeps the last entry. With Saxon configured, it evaluates XPath 2.0 and 3.0 **[XPATH2]**.

5 Processing Model

A processor turns a template and data into an instance document through a fixed sequence of steps. Steps 2 to 6 are the core of OpenDoPE. The others are optional.

Table 4 – Processing steps

#	Step	Effect	Section
1	Insert data	Replace the content of the data part with the runtime XML.	Error! Reference source not found.
2	Fetch components	Replace component references with the referenced documents.	Error! Reference source not found.
3	Preprocess	Evaluate conditions and expand repeats, in document order, recursively.	Error! Reference source not found., Error! Reference source not found.
4	Repair integrity	Fix structures that repeats and conditions can break.	Error! Reference source not found.
5	Bind	Write values into bound content controls, and resolve position conditions.	Error! Reference source not found.
6	Repair after binding	Fix structures that binding can break.	Error! Reference source not found.
7	Finish	Apply a user XSLT to controls tagged <code>od:finish</code> .	Error! Reference source not found.
8	Remove content controls	Unwrap content controls, keeping their content.	Error! Reference source not found.
9	Remove parts	Delete the data, XPaths and Conditions parts.	Error! Reference source not found.

REQ-015 A processor **MUST** perform the steps it performs in the order of the table above.

REQ-016 A processor **MUST** apply steps 3 to 8 to the main document part and to every header and footer part related from it.

NOTE Component fetching applies only to the main document part. Footnotes, endnotes and comments are not processed.

Steps 3 and 5 are separate because a consumer such as Word performs the equivalent of step 5 itself when it opens a document with live data bindings. A document that has been through steps 1 to 4 is therefore already usable in Word. Step 5 is needed when the result will be converted to another format, such as PDF, without passing through Word.

6 Data Binding

6.1 Resolving the Value

A content control is bound if it has an `od:xpath` tag, a `w:dataBinding` or a `w15:dataBinding` [MS-DOCX].

- REQ-017 A processor **MUST** take a control's expression from its `od:xpath` entry where the tag names one that exists, and otherwise from its `w:dataBinding` or `w15:dataBinding`.
- REQ-018 A processor **MUST** use the XPath string value of the expression, evaluated against the part named by the store item ID.
- REQ-019 A processor **MUST** remove leading and trailing white space from the value.
- REQ-020 Where an expression begins with `local-name`, a processor **MUST** decode escapes of the form `_xHHHH_` in the value to the characters they encode.

NOTE The `_xHHHH_` escape lets data whose names are not valid XML names, such as JSON keys converted to XML, round-trip through element names.

6.2 Plain Text Values

The value replaces the control's content. The processor keeps the control's structure: a control whose content is a paragraph, cell, row or table keeps that element and its properties, and the value goes into its first paragraph. A run-level control receives runs directly.

- REQ-021 A processor **MUST** apply the `w:rPr` of the control's `w:sdtPr` to the runs it generates.
- REQ-022 Where `w:text/@w:multiLine` is true, a processor **MUST** split the value at line feeds, carriage returns and form feeds, and separate the lines with `w:br`.
- REQ-023 Where the control is not multi-line, a processor **MUST** remove line breaks from the value.
- REQ-024 Where the value is empty, a processor **MUST** insert placeholder text styled `PlaceholderText` and set `w:showingPlcHdr` on the control.
- REQ-025 A processor **MUST** remove `w:placeholder` from the properties of a control it has bound.

NOTE docx4j reads the placeholder run from the resource named by the property `docx4j.model.datastorage.placeholder`, falling back to "Click here to enter text."

6.3 Hyperlinks

Hyperlink detection is optional and is enabled by naming a character style for hyperlinks. When it is enabled, text in the value that starts with `http://`, `https://` or `mailto:` becomes a `w:hyperlink` with an external relationship. The link ends at the next space. Text before the link remains a plain run.

- REQ-026 Where the bound content contains a `w:hyperlink`, a processor **MUST** remove `w:dataBinding` and `w:text` from the control's properties.

NOTE Word 2007 cannot open a document in which a plain text data-bound control contains a hyperlink.

6.4 Typed Content Controls

Some Word content control types need their own treatment. These rules apply to controls with a `w:dataBinding`.

Table 5 – Typed content controls

Control	Binding behaviour
Checkbox <code>w14:checkbox</code>	The value <code>true</code> or <code>1</code> checks the box. Anything else clears it. The processor sets <code>w14:checked</code> and replaces the content with the checked or unchecked glyph.
Date <code>w:date</code>	The value is parsed as an <code>xs:dateTime</code> or <code>xs:date</code> and formatted with <code>w:dateFormat</code> . An unparseable value is shown as the current date in red, as Word does.
Picture <code>w:picture</code>	The value is base64 image data [RFC2045]. Where the content has an <code>a:blip</code> , only its <code>r:embed</code> is replaced, so size and position are kept. Otherwise a new inline image is generated, fitted to the extent in the content.

REQ-027 Where a checkbox or date control’s data part cannot be read, a processor **MUST** insert a visible marker instead of a value.

NOTE docx4j inserts the text “[missing!]”. Base64 decoding is lenient and ignores characters outside the alphabet, including line breaks.

7 Conditions

7.1 The Conditions Part

A control tagged `od:condition=ID` is kept when condition ID is true and dropped when it is false. Conditions are defined in the Conditions part. A condition is a tree of boolean operators whose leaves refer to XPath entries or to other conditions.

```
<conditions xmlns="http://opendope.org/conditions">
  <condition id="c1">
    <xpathref id="x5"/>
  </condition>
  <condition id="c2">
    <and>
      <conditionref id="c1"/>
      <not><xpathref id="x6"/></not>
    </and>
  </condition>
</conditions>
```

Table 6 – Condition elements

Element	Content	Value
condition	exactly one of xpathref, and, or, not	The value of its child. Carries @id and optional @name, @description, @comments, @source.
xpathref	empty; @id names an XPath entry	The XPath entry converted to boolean (Section Error! Reference source not found.).
conditionref	empty; @id names a condition	The value of that condition.
and	one or more operands	True when every operand is true. Evaluation stops at the first false operand.
or	one or more operands	True when any operand is true. Evaluation stops at the first true operand.
not	exactly one operand	The negation of the operand.

An operand is any of xpathref, conditionref, and, or or not. A condition cannot contain a conditionref directly, but any operator inside it can.

REQ-028 Each condition/@id **MUST** be unique within the Conditions part.

REQ-029 A processor **MUST** refuse to process a template in which following conditionref elements revisits a condition or reaches a missing condition.

REQ-030 A processor **MUST** treat a tag that names a missing condition as an error.

NOTE Before the acyclic check was added, a crafted document with self-referencing conditions exhausted the stack of the thread processing it.

7.2 Boolean Conversion

How an XPath entry becomes a boolean depends on the boolean conversion mode. Three modes are defined.

Table 7 – Boolean conversion modes

Mode	Rule	Examples
Java	The string value is true only if it equals <code>true</code> , ignoring case.	"True" is true. "1", "yes" and "false" are false.
XPath 1.0	The XPath <code>boolean()</code> rules [XPATH1]. A node-set is true if non-empty. A string is true if non-empty.	"false" is true. <code>/a/b</code> is true if b exists.
XPath 2.0	The effective boolean value. A string cast to <code>xs:boolean</code> accepts only <code>true</code> , <code>false</code> , <code>1</code> and <code>0</code> .	"false" is false. "yes" is an error.

The same expression can be true in one mode and false in another, so the authoring tool and the processor must agree on the mode. The XPaths part therefore declares the mode its expressions were written for, in the `booleanConversion` attribute of its root element. The values are `java`, `xpath1` and `xpath2`.

```
<xpaths xmlns="http://opendope.org/xpaths" booleanConversion="xpath2">
```

- REQ-031** A template **MUST** declare its boolean conversion mode in `xpaths/@booleanConversion`.
- REC-003** An authoring tool **SHOULD** support XPath 2.0, and an authoring tool that does **SHOULD** author in the `xpath2` mode by default.
- REQ-032** When the XPaths part declares a boolean conversion mode, a processor **MUST** evaluate every condition in that mode, or refuse to process the template if it cannot, for example because `xpath2` is declared and no XPath 2.0 evaluator is available.

A template that predates this version has no declaration. For such a template the mode is chosen by processor configuration, and the Java mode is the default.

- REC-004** New templates **SHOULD** be written for the XPath 2.0 mode, using expressions that are already boolean, such as `/invoice/total > 1000` or `count(/invoice/items/item) > 0`.

NOTE In docx4j the property `opendope.conditions.XPathref.XPathBoolean` selects the mode for a template that declares none: `false` for Java, `true` or `cast1` for XPath 1.0, and `cast2` for XPath 2.0, which requires Saxon. docx4j 17.1.1 does not read the attribute; the property applies to every template.

7.3 Effect of a Condition

- REQ-033** When a condition is true, a processor **MUST** keep the control and its content, and continue processing inside it.
- REQ-034** When a condition is false, a processor **MUST** remove the control's content from the instance document.

A processor that supports reverting (Section **Error! Reference source not found.**) keeps an empty marker in place of the removed control. Otherwise it removes the control altogether.

- REQ-035** A marker for a false condition **MUST** have its `od:condition` parameter renamed `od:resultConditionFalse`, keeping its value and the other parameters.
- REQ-036** A marker **MUST** carry `w:lock` with the value `sdtContentLocked`, and no `w:dataBinding`.
- REQ-037** Where the removed content was a table cell, a processor **MUST** leave a cell with its cell properties and one empty paragraph, so the table stays rectangular.

Removing a control that wraps a whole row removes the row. Where removal leaves a cell or row invalid, the integrity step repairs it (Section **Error! Reference source not found.**).

8 Repeats

8.1 Expanding a Repeat

A control tagged `od:repeat=ID` is copied once for each node that XPath's entry ID selects. Anything can be repeated: paragraphs, runs, table rows, table cells, or a mixture.

- REQ-038** A repeat expression **MUST** select the elements to be repeated, not their container.

EXAMPLE For an invoice, the repeat expression is `/invoice[1]/items[1]/item`, not `/invoice[1]/items[1]`.

The processor first normalises the expression into the *repeat base* by removing one trailing `/` or one trailing `[1]`. It then counts the nodes the repeat base selects.

- REQ-039** A processor **MUST** produce exactly as many repeat instances as there are nodes selected by the repeat base.
- REQ-040** When the count is zero, a processor **MUST** remove the control's content, applying the rules for a false condition with `od:resultRepeatZero` in place of `od:resultConditionFalse`.

NOTE This is different to how Word's Repeating Sections handles a count of zero (it keeps one stale item). See **REQ-053** in Section **Error! Reference source not found.**

When the count is *n* greater than zero, the processor makes *n* copies of the control. Before copying, it rewrites the tag of the control:

- the `od:repeat` parameter is renamed `od:rptd`, keeping its value;
- a parameter `od:RptOcc` is appended whose value numbers this occurrence, counting from 1 across the document;
- any `w:dataBinding` on the repeat control itself is removed.

- REQ-041** All instances of one occurrence **MUST** carry identical tags.
- REQ-042** A processor **MUST** keep the content control `w:id` of the first instance and give every other instance, and every content control inside an instance, a new `w:id`.

REQ-043 A processor **MUST** rename and renumber bookmarks inside repeat instances so that bookmark names and IDs remain unique.

NOTE Keeping the first instance's identifier lets a reverter match the expanded repeat to the control in the template.

Instances are then processed in turn, so conditions and repeats nested inside a repeat are evaluated for each instance, against that instance's data.

REQ-044 A processor **MUST** process conditions, repeats and position conditions inside each repeat instance after contextualising their expressions for that instance.

8.2 Contextualising Expressions

Templates are written against a sample: expressions inside a repeat refer to the first item, for example `/invoice[1]/items[1]/item[1]/name`. For instance `i`, the processor rewrites every expression in the instance that refers to the repeat base, inserting the position predicate `[i]` immediately after the repeat base.

Table 8 – Contextualisation for repeat base `/doc/items/item`` and instance 3

Expression in template	Expression in instance 3
<code>/doc/items/item/name</code>	<code>/doc/items/item[3]/name</code>
<code>/doc/items/item[1]/name</code>	<code>/doc/items/item[3][1]/name</code>
<code>string(/doc/items/item/qty) > 0 and boolean(/doc/items/item/tax)</code>	<code>string(/doc/items/item[3]/qty) > 0 and boolean(/doc/items/item[3]/tax)</code>
<code>/doc/items/item/notes/note[last()]</code>	<code>/doc/items/item[3]/notes/note[last()]</code>
<code>/doc/items/item[last()]/name</code>	unchanged
<code>/doc/items/item[position()=1]/name</code>	unchanged
<code>count(/doc/items/item)</code>	unchanged
<code>/doc/customer/name</code>	unchanged

REQ-045 A processor **MUST** contextualise every location path in an expression that begins with the repeat base.

REQ-046 A processor **MUST NOT** change any other part of the expression.

REQ-047 Where the predicates immediately following the repeat base use `position()` or `last()`, a processor **MUST** leave that path unchanged.

REQ-048 Where an expression is `count()` applied to exactly the repeat base, a processor **MUST** leave it unchanged.

NOTE A numeric predicate such as `[1]` at the repeat step is kept after the inserted predicate. `item[3][1]` selects the same node as `item[3]`.

Because tags refer to entries by ID, contextualising creates new entries instead of editing shared ones:

- For a bound control, a new XPaths entry is created with ID `X_o_i`, where X is the original ID, o the occurrence number and i the zero-based instance index. The control's `od:xpath` and `w:dataBinding` are updated.
- For a nested repeat, or a control tagged `od:ContentType`, `od:Handler` or `od:progid`, a new XPaths entry is created in the same way and the tag updated.
- For a condition, a copy of the condition is created with ID `C_i`, whose `xpathref` and `conditionref` descendants refer to contextualised copies with IDs of the form `X_i`.

REQ-049 A processor **MUST** write the added XPaths entries and conditions to the XPaths and Conditions parts of the instance document.

8.3 Position Conditions

Some content depends on an instance's position rather than its data, such as the word “and” before the last item of a list. A control inside a repeat tagged `od:RptPosCon=ID` is kept only if the expression in XPaths entry ID is true for the enclosing instance.

The expression is written as an XPath predicate using `position()` and `last()`, for example `position()=last()-1`. It is evaluated during the binding step, after all repeats have been expanded. The entry's store item ID is required by the schema but not used.

REQ-050 A processor **MUST** evaluate a position condition against the nearest enclosing repeat instance, where position is that instance's one-based rank among its sibling instances with an identical tag, and last is the number of those instances.

REQ-051 When a position condition is false, a processor **MUST** omit the control and its content.

EXAMPLE Items rendered as “apples, bananas and cherries” use three run-level controls inside a run-level repeat: the item name, a control containing “,” with `position()<last()-1`, and a control containing “ and ” with `position()=last()-1`.

NOTE Matching on identical tags, including `od:RptOcc`, keeps two occurrences of the same repeat in one paragraph or table apart.

8.4 Word Repeating Sections

Word 2013 introduced its own repeating section content control **[MS-DOCX]**. It is a control with `w15:repeatingSection` and a `w15:dataBinding` to the container, whose content is a single `w15:repeatingSectionItem` control. It needs no XPaths part.

REQ-052 A processor that supports repeating sections **MUST** replace the item control with one copy per node selected by the `w15:dataBinding`, contextualising descendant bindings as for an OpenDoPE repeat.

REQ-053 When the count is zero, a processor **MUST** leave the content unchanged and set the tag to `w15:resultRepeatZero=true`.

REQ-054 The binding step **MUST NOT** alter a control tagged `w15:resultRepeatZero`.

REQ-055 After expansion, the integrity step **MUST** unwrap repeating section and repeating section item controls, keeping their content.

NOTE Leaving the content unchanged on zero matches Word, which keeps one stale item. OpenDoPE repeats are recommended where zero items must yield no content, and where documents are edited in Word 2010, which strips w15 properties.

9 Rich Content

Word data binding only carries plain text. The following tag parameters let a rich text control receive formatted content from the data part. Such a control has an `od:xpath` tag and no `w:dataBinding`.

9.1 XHTML

A control tagged `od:ContentType=application/xhtml+xml` receives XHTML **[XHTML1]**. The data element holds the XHTML as escaped text.

```
<case1><div><p>Hello
<b>world</b></p><p>Again</p></div></case1>

<xpath id="x7">
  <dataBinding storeItemID="{8B049945-9DFE-4726-9DE9-CF5691E53858}"
    xpath="/case1[1]"/>
</xpath>

<w:sdt>
  <w:sdtPr>
    <w:tag w:val="od:xpath=x7&od:ContentType=application/xhtml+xml"/>
  </w:sdtPr>
  <w:sdtContent><w:p></w:sdtContent>
</w:sdt>
```

REQ-056 The value of an XHTML control **MUST** be well-formed XML with a single root element.

REQ-057 A template **MUST** place an XHTML control whose XHTML contains block elements at block level or inside a table cell.

REQ-058 A processor **MUST** either convert the XHTML to WordprocessingML in place of the control's content, or insert it as a `w:altChunk` of type XHTML for the consumer to convert.

When converting, the processor resolves two sources of formatting: the control's own `w:rPr`, and the CSS in the XHTML. By default the control's formatting is the baseline and the XHTML overrides it. Existing Word styles are used where an XHTML `class` attribute names one.

REC-005 A processor **SHOULD** discard, with a warning, block content produced for a run-level control.

NOTE docx4j converts with docx4j-ImportXHTML [**IMPORTXHTML**] when it is present, and otherwise inserts an altChunk that Word resolves on opening. The properties docx4j.model.datastorage.BindingTraverser.XHTML.PrioritiseRPr and ...XHTML.RunFormatting, ...ParagraphFormatting and ...TableFormatting adjust the formatting rules. Lists under class-only formatting need a class on each list level, or they lose their numbering.

9.2 Word Fragments

A control tagged `od:progid=Word.Document` receives a Word document serialised as Flat OPC XML (`pkg:package`) and escaped as text in the data element. This binds tables, images and styled content, and works with consumers that predate Word 2013's own support for binding such content.

REQ-059 A processor **MUST** replace the control's content with a `w:altChunk` whose target part holds the Flat OPC XML, or with the equivalent WordprocessingML.

REQ-060 A template **MUST** place a Word fragment control at block level or inside a table cell.

NOTE Content controls inside a fragment are not bound by the processor. Nesting fragment controls is not supported.

9.3 Pictures

Word forbids floating picture content controls, so an image that must float is placed in a rich text control tagged `od:Handler=picture`. The data element holds base64 image data.

REQ-061 For a control tagged `od:Handler=picture` without a `width` parameter, a processor **MUST** replace only the `r:embed` of the first `a:blip` in the content with a relationship to a new image part.

REQ-062 For a control tagged `od:Handler=picture` with `width=auto` or `width=N`, a processor **MUST** replace the content with a new inline image at its natural size, scaled down to fit N twentieths of a point or the page's text width, whichever is smaller.

EXAMPLE `od:xpath=logo&od:Handler=picture&width=4500` fits the logo into a width of 4500 twentieths of a point, about 7.9 cm.

10 Components

A component is a separate document included in the template when the instance document is generated. A control tagged `od:component=ID` is replaced by the document that Components part entry ID refers to.

```
<components xmlns="http://opendope.org/components">
  <component id="comp1" iri="clauses/confidentiality.docx"/>
</components>
```

The `iri` identifies the component. Its scheme and resolution are defined by the processing application, not by this specification.

- REQ-063** A processor **MUST** fetch and insert components before evaluating conditions and repeats.
- REQ-064** A processor **MUST** evaluate a component's bindings in the context given by the tag parameter `od:context=XPATHID` where present, and otherwise in the context of the nearest enclosing repeat instance, and otherwise at the document root.
- REQ-065** When a component brings OpenDoPE parts, a processor **MUST** merge its entries into the host's parts, discarding entries whose IDs the host already uses.

Component processing is not recursive: component references inside a fetched component are left unprocessed. Components are supported only in the main document part.

NOTE Because components are fetched before conditions are evaluated, a component inside a false condition is fetched and then removed with the condition. The parameters `od:continuousBefore` and `od:continuousAfter` from version 2.3, which asked for continuous section breaks around a component, are reserved but not currently acted on.

NOTE In docx4j component processing is off by default. It is enabled by `docx4j.model.datastorage.OpenDoPEHandlerComponents.enabled`, fetches through a `DocxFetcher`, and needs the Enterprise MergeDocx functionality to merge the fetched content. Without it the component stays in the document as an `altChunk`.

11 After Binding

11.1 Integrity Repair

Copying and removing content can produce markup that Word refuses to open. A processor repairs it in two places.

After preprocessing:

- Comment ranges and references, footnote references and endnote references whose ID has already occurred are removed, because copies of repeated content share IDs.
- A content control wrapping a table cell must contain a cell with at least one paragraph. One is created if needed.
- A block-level content control inside a table cell must contain at least one paragraph. One is created if needed.
- Word repeating section and item controls are unwrapped (Section **Error! Reference source not found.**).

After binding:

- A bookmark start inside a plain text content control is moved to just before the control.

- REQ-066** A processor **MUST NOT** emit an instance document that violates the constraints repaired by these steps.

11.2 Finishing

Formatting that depends on data, such as shading a row whose amount is negative, can be expressed with conditions. It is often neater as a transformation. A control tagged

`od:finish=NAME` is a target for the finishing step, which applies a user-supplied XSLT to each part.

The stylesheet receives parameters for the package, the part, the data parts, the XPath map and a map of user parameters keyed by template name. It typically matches controls by the value of their `od:finish` parameter and calls helper functions to read bound values.

REC-006 A processor **SHOULD** skip the finishing step when no stylesheet is configured.

11.3 Removing Content Controls and Parts

Content controls and OpenDoPE parts can be removed from the instance document when it will not be processed again. Removal keeps a control's content and discards the control.

Table 9 – Removal modes

Mode	Controls removed
All	Every content control. A bound control whose content has no text is removed with its content, except that a cell, drawing or altChunk inside it is kept. This is the default.
All but placeholders	Every content control except those with <code>w:showingPlcHdr</code> , which are kept whole.
All but placeholder controls	Every content control. The content of placeholder controls is kept.
Default	Condition and repeat controls only.
Named	The kinds listed: <code>xpath</code> , <code>condition</code> , <code>repeat</code> and <code>empty</code> .

REQ-067 In every mode that removes condition or repeat controls, a processor **MUST** remove `od:resultConditionFalse` and `od:resultRepeatZero` markers together with their content.

REQ-068 After removal, a processor **MUST** remove table rows left without cells, and give every cell left without a paragraph an empty paragraph.

Removing parts deletes the data part, the XPath part and the Conditions part. Other custom XML parts are kept.

NOTE Removed controls cannot be reverted, and Word can no longer update their values from the data.

11.4 Reverting

Reverting turns an instance document back into a template, so that it can be regenerated with fresh data while keeping manual edits made outside conditions and repeats.

REQ-069 Reverting **MUST** use both the original template and the instance document.

REQ-070 An instance document to be reverted **MUST** still contain its condition and repeat controls and markers.

- REQ-071** A reverter **MUST** replace each top-level group of repeat instances, and each condition control or marker, with the control from the template that has the same `w:id`.
- REQ-072** A reverter **MUST** treat adjacent repeat instances with different `od:RptOcc` values as separate repeats.
- REC-007** A reverter **SHOULD** report failure when the numbers of condition and repeat controls in the result differ from the template.

NOTE Edits inside a condition or repeat are lost when reverting. Comparing the regenerated document with the previous instance recovers them.

11.5 Updating Data from the Document

Word copies edits in a data-bound plain text control back to the data part. It does not do this for rich content. A processor may offer the reverse operation for controls tagged `od:progid=Word.Document`: it serialises the control's content as escaped Flat OPC XML and writes it to the bound element.

12 Security Considerations

Templates and data may come from untrusted sources. Processors face the following risks.

- Unbounded recursion. Cyclic condition references would recurse without limit. **REQ-029** closes this.
- Resource exhaustion. A repeat over a large node-set, nested repeats, or expensive XPath expressions can consume much time and memory. Applications should limit document size and processing time.
- External entities. Custom XML parts and Flat OPC fragments are XML, and may declare external entities or DTDs.
- Fetching components. A component IRI can name a local file or network resource. The application decides which IRIs its component fetcher resolves.
- Injected content. XHTML and Word fragments in data can carry hyperlinks, images and `altChunks` into the instance document.

- REQ-073** A processor **MUST NOT** resolve external entities when parsing custom XML parts or bound XML content.

13 Conformance

This specification has three conformance targets.

A *conforming template* is a WordprocessingML package that satisfies every requirement whose subject is a template, and whose OpenDoPE parts are valid against the schemas published with the reference implementation.

A *conforming authoring tool* satisfies every requirement whose subject is an authoring tool.

- REQ-074** A conforming authoring tool **MUST** produce conforming templates.

A *conforming processor* satisfies every requirement whose subject is a processor for the features it claims. The core features are data binding (Section **Error! Reference source not found.**), conditions (Section **Error! Reference source not found.**), repeats and position conditions (Section **Error! Reference source not found.**), and integrity repair.

REQ-075 A conforming processor **MUST** implement the core features.

The optional features are typed content controls, rich content, components, Word repeating sections, finishing, removal and reverting.

REQ-076 A processor that does not implement an optional feature **MUST** leave the controls using it unchanged.

REC-008 A processor **SHOULD** report each control it left unprocessed for lack of an optional feature.

Appendix A Tag Parameter Reference (Normative)

Table 10 – Tag parameters

Parameter	Value	Set by	Meaning
od:xpath	XPaths entry ID	template	Binds the control (Section Error! Reference source not found.).
od:condition	condition ID	template	Keeps the control only if the condition is true (Section Error! Reference source not found.).
od:repeat	XPaths entry ID	template	Repeats the control per selected node (Section Error! Reference source not found.).
od:RptPosCon	XPaths entry ID	template	Keeps the control only for matching repeat positions (Section Error! Reference source not found.).
od:component	component ID	template	Replaces the control with a component (Section Error! Reference source not found.).
od:context	XPaths entry ID	template	Context for a component's bindings.
od:continuousBefore, od:continuousAfter	true	template	Reserved. Continuous section break around a component.
od:ContentType	application/xhtml+xml	template	Binds escaped XHTML (Section Error! Reference source not found.).
od:progid	Word.Document	template	Binds escaped Flat OPC XML (Section Error! Reference source not found.).
od:Handler	picture	template	Binds base64 image data into a rich text control (Section Error! Reference source not found.).
width	auto or integer	template	Maximum picture width in twentieths of a point, with od:Handler=picture.
od:finish	template name	template	Target for the finishing XSLT (Section Error! Reference source not found.).
od:narrative	any	template	Reserved.
od:rptd	XPaths entry ID	processor	Marks an expanded repeat instance.
od:RptOcc	integer	processor	Numbers the occurrence of an expanded repeat.
od:resultConditionFalse	condition ID	processor	Marker left for a false condition.

Parameter	Value	Set by	Meaning
od:resultRepeatZero	XPaths entry ID	processor	Marker left for a repeat with no items.
w15:resultRepeatZero	true	processor	Marks a Word repeating section with no items.

REQ-077 A template **MUST NOT** contain the parameters marked as set by the processor.

Appendix B docx4j Implementation Notes (Informative)

docx4j [DOCX4J] implements this specification in the package `org.docx4j.model.datastorage`.

B.1 Running the Pipeline

The `Docx4J.bind` facade runs the whole pipeline. Flags select the steps; `FLAG_NONE` selects all of them.

```
WordprocessingMLPackage pkg = Docx4J.load(new File("invoice.docx"));
try (InputStream data = new FileInputStream("invoice-data.xml")) {
    Docx4J.bind(pkg, data,
        Docx4J.FLAG_BIND_INSERT_XML | Docx4J.FLAG_BIND_BIND_XML);
}
Docx4J.save(pkg, new File("OUT_invoice.docx"));
```

Table 11 – Bind flags

Flag	Steps	Effect
<code>FLAG_BIND_INSERT_XML</code>	1	Replace the data part with the supplied XML.
<code>FLAG_BIND_BIND_XML</code>	2 to 6	Components (when enabled), preprocessing, integrity, binding, integrity after binding.
<code>FLAG_BIND_REMOVE_SDT</code>	8	Remove content controls, using the configured removal mode.
<code>FLAG_BIND_REMOVE_XML</code>	9	Remove the data, XPath expressions and Conditions parts.

An overload taking a `DocxFetcher`, an `XsltProvider`, a stylesheet name and finisher parameters adds components and finishing. The steps can also be run individually, which lets a caller inspect intermediate results:

```
pkg = new OpenDoPEHandlerComponents(pkg).fetchComponents(); // step 2
OpenDoPEHandler odh = new OpenDoPEHandler(pkg);
pkg = odh.preprocess(); // step 3
new OpenDoPEIntegrity().process(pkg); // step 4
BindingHandler bh = new BindingHandler(pkg);
bh.setStartingIdForNewBookmarks(odh.getNextBookmarkId());
bh.applyBindings(); // step 5
new OpenDoPEIntegrityAfterBinding().process(pkg); // step 6
new RemovalHandler().removeSDTs(pkg,
    RemovalHandler.Quantifier.ALL_BUT_PLACEHOLDERS); // step 8
```

`OpenDoPEReverter(template, instance).revert()` reverts an instance.
`UpdateXmlFromDocumentSurface` writes rich content back to the data part.

B.2 Configuration Properties

Table 12 – docx4j.properties keys

Key	Effect
<code>docx4j.model.datastorage.BindingHandler.Implementation</code>	Binding traverser: <code>BindingTraverserNonXSLT</code> (default, fastest), <code>BindingTraverserXSLT</code> (customisable through <code>bind.xslt</code>) or <code>BindingTraverserStAX</code> (lowest memory). All three have the same features.
<code>opendope.conditions.XPathref.XPathBoolean</code>	Boolean conversion mode for a template that declares none: <code>false</code> , <code>true/cast1</code> or <code>cast2</code> .
<code>docx4j.openpackaging.parts.XmlPart.xpath2.typechecking</code>	With Saxon, cast operands of mixed-type comparisons: <code>strict</code> , <code>cast1</code> or <code>cast2</code> .
<code>docx4j.model.datastorage.OpenDoPEHandler.Components.enabled</code>	Enable component processing. Default <code>false</code> .
<code>docx4j.model.datastorage.OpenDoPEReverter.Supported</code>	Leave markers for false conditions and empty repeats. Default <code>true</code> .
<code>docx4j.model.datastorage.RemovalHandler.Quantifier</code>	Removal mode used by the facade: <code>ALL</code> (default), <code>ALL_BUT_PLACEHOLDERS</code> , <code>ALL_BUT_PLACEHOLDERS_CONTENT</code> , <code>DEFAULT</code> or <code>NAMED</code> .
<code>docx4j.model.datastorage.placeholder</code>	Resource holding the placeholder run.
<code>docx4j.model.datastorage.BindingTraverser.XHTML.PrioritiseRPr</code>	When <code>true</code> , the control's formatting overrides the XHTML's CSS.
<code>docx4j.model.datastorage.BindingTraverser.XHTML.RunFormatting and ParagraphFormatting, TableFormatting</code>	XHTML importer formatting options, such as <code>CLASS_TO_STYLE_ONLY</code> .

B.3 Extension Points

The following static hooks on `BindingHandler` apply to all subsequent binds in the JVM.

- `setHyperlinkResolver`, with `BindingHyperlinkResolver` or a subclass: decides what text is a link and how the link is built.
`getHyperlinkResolver().setHyperlinkStyle("Hyperlink")` enables detection.
- `setValueInserterPlainText`, with `ValueInserterPlainText`: turns a bound value into runs. `ValueInserterPlainTextForOpenAPI3` adds bookmarks and internal links for OpenAPI documents.
- `setXHTMLImporterCustomizer`, with `XHTMLImporterCustomizer`: configures the XHTML importer per control.

`DocxFetcher` resolves component IRIs. `XsltProvider` supplies finishing stylesheets.

The `migration` package converts templates built on merge fields, legacy form fields or `${variable}` placeholders into data-bound content controls with OpenDoPE parts.

Appendix C Changes from Version 2.3 (Informative)

Table 13 – Changes since version 2.3

Area	Change
Processing model	Components are fetched before conditions and repeats, may sit anywhere in the main document, take a context, and are not recursive. Integrity repair, finishing, removal and reverting are specified.
Repeats	Instance tags use <code>od:rptd</code> and <code>od:RptOcc</code> . Cloned XPath IDs include the occurrence. Expressions with <code>position()</code> or <code>last()</code> at the repeat step address a specific item. Zero repeats leave an <code>od:resultRepeatZero</code> marker.
Position conditions	New <code>od:RptPosCon</code> parameter.
Conditions	<code>conditionref</code> operand. Cyclic or dangling references are rejected. Boolean conversion modes are defined and the XPaths part declares which one it uses. A false condition leaves an <code>od:resultConditionFalse</code> marker.
Binding	Rules for trimming, multi-line text, placeholders, run properties, hyperlinks, checkbox, date and picture controls, and <code>w15:dataBinding</code> .
Rich content	New <code>od:ContentType=application/xhtml+xml</code> , <code>od:progid=Word.Document</code> and <code>od:Handler=picture</code> with <code>width</code> .
Word repeating sections	Supported as an alternative to OpenDoPE repeats.
Data part	Selection rules defined, including the fallback to a sole candidate part.
Tag syntax	Grammar defined. <code>od:</code> keys outside the vocabulary are ignored.
XPaths part	The root element declares the specification version, the boolean conversion mode and the authoring tool. A processor refuses templates from a newer version.

Appendix D Acknowledgements (Informative)

The OpenDoPE conventions were developed by Plutext with contributions from docx4j users. Karsten Tinnefeld contributed content control removal. Issue reports in the docx4j project, including issue 690 and discussion 691, shaped the handling of repeated occurrences and position predicates.

Appendix E Revision History (Informative)

Table 14 – Revision history

Version	Date	Changes
1	2010	Initial conventions, implemented in docx4j 2.5.0.
2.2	2010-11-16	Short IDs in tags, boolean conditions, interactive processing, components.
2.3		Published at opendope.org. Repeat XPaths point to the repeated element.
3.0	2026-09-14	Working Draft 1 describing docx4j 17.1.1.
3.0	2026-09-18	Working Draft 2. Specification version, boolean conversion mode and authoring tool declared in the XPaths part. Authoring tool added as a conformance target.